

AGNO 学习笔记 · CHAPTER 1

第一个 Agent

20 行代码，让模型从「会聊天」变成「会干活」。

EXAMPLES/01_FIRST_AGENT · DEEPSEEK · AGNO SDK

直接调模型 API，你得到的只是半个大脑

- ▶ 无状态：每次请求都是陌生人见面，上次说了什么全忘了
- ▶ 只会说，不会做：能生成文本，但不能查股价、不能搜网页、不能碰你的世界
- ▶ 没有规矩：同样的输入，输出的格式和风格全看模型心情

Agent = 在裸模型外面包一层 控制循环，补上状态、工具和规矩。

“Agents are a **stateful control loop** around a **stateless model**.”

— docs.agno.com/agents/overview (官方原话)

stateless model

模型本身无记忆

状态是 Agent 给的，不是模型长的

control loop

推理 → 调工具 → 看结果 → 再推理

循环直到能回答为止

guided by instructions

循环听指令指挥

instructions 是方向盘

最小 Agent, 全部家当就这些

```
agent = Agent(  
    name="助手",  
    model=DeepSeek(  
        id=getenv("DEEPSEEK_MODEL_ID",  
            "deepseek-v4-flash"),  
        api_key=api_key,  
    ),  
    description="一个友好的AI助手",  
    markdown=True,  
)  
  
agent.print_response("你好, 请介绍一下自己", stream=True)
```

- ▶ **model** 是大脑: DeepSeek 适配器, 也支持 "openai:gpt-5.5" 字符串简写
- ▶ **description** 写人设, 会进入 system prompt
- ▶ **markdown=True** 让终端输出带格式
- ▶ **stream=True** 边生成边打印, 不用干等

Agent() 的旋钮都在哪

参数	干什么	你的代码里的例子
model	大脑：模型 + key + 超时/重试	DeepSeek(id="deepseek-v4-flash", api_key=...)
name / description	身份名片，拼进 system prompt	name="金融分析师"
instructions	行为规矩，字符串或列表	["使用表格展示数据", "只输出分析报告"]
markdown	终端渲染 Markdown 输出	markdown=True

调 Agent 行为，先动 instructions，再考虑换模型。

instructions 是 Agent 的岗位说明书

```
instructions=[  
    "必须先使用 MCP 工具查询 Agno 官方文档，再回答问题。",  
    "回答中提供相关文档路径或链接。",  
] # examples/01_first_agent/03_mcp.py
```

- ▶ 可以传列表，一条一条写，比一大段散文好维护
- ▶ 写「必须 / 不要」比写「尽量 / 最好」有效得多
模型对强约束的服从度明显更高
- ▶ 输出不稳定时，**第一站**是改这里，而不是换模型

三种把 Agent 跑起来的姿势

`print_response()`

直接打印到终端

`stream=True` 流式；调试期最顺手

`run()`

返回 `RunOutput` 对象

`response.content` 取文本，交给后续 Python 逻辑

`aprint_response()`

异步版

配 MCP 等异步工具时必须用它

```
# examples/01_first_agent/05_weather.py - 把输出接回自己的程序
response = weather_agent.run("纽约一年中每个月的天氣如何? ")
print(response.content)
```

expected_output 只是提示，不是校验

⚠ 你踩过的坑

定义了 `WeatherData(BaseModel)`，却没有传给 Agent —— 它只是一个摆设。靠 `expected_output="包含月份、季节和平均温度的表格"` 拿到的表格，没有任何 Pydantic 校验。

✓ 怎么选

要机器可读、可校验的结构 → 用
`output_schema=WeatherData`

只要格式上的口头提示 → `expected_output` 就够
结构化输出在 ch3 还会深入

提示词管不住格式时，别再「多写几句」，→ 上 `output_schema`。

一行 `tools=[...]`，Agent 就能碰真实世界

```
agent_full = Agent(  
    name="金融分析师",  
    role="专业的股票市场分析师",  
    model=DeepSeek(...),  
    tools=[YFinanceTools()],    # ← 就这一行，会查股价了  
    markdown=True,  
) # examples/01_first_agent/04_finacial.py
```

⚠ 代码注释会骗人

文件里写着 `# All functions enabled by default`，但 README 实测：当前版本 `YFinanceTools()` 默认只启用 `get_current_stock_price`。要做完整分析得用 `YFinanceTools(all=True)` 或显式启用函数。

工具是 ch2 的主菜，这里先记住：给工具 = 给能力。

MCP: 把别人的服务变成你的工具

```
async with MCPTools(  
    transport="streamable-http",  
    url="https://docs.agno.com/mcp",  
) as mcp_tools:  
    agent = Agent(tools=[mcp_tools], ...)  
    await agent.aprint_response("...", stream=True)
```

- ▶ Agno 官方文档站本身就是 MCP 服务
Agent 可先查文档再回答你
- ▶ 用 **async with** 管理连接生命周期
- ▶ 异步连接必须配 **aprint_response**
改回同步 `print_response` 会炸 —— 你 README 里记过

同一个 Agent, 包一层 AgentOS 就是生产 API

```
agent_os = AgentOS(  
    id="web-search-os",  
    agents=[web_agent],  
)  
app = agent_os.get_app()  
agent_os.serve(app=app, host="localhost", port=7777)
```

- ▶ AgentOS 是 FastAPI 运行时
白得 /docs Swagger 文档
- ▶ 浏览器打开 os.agno.com 连上本地 7777
就有现成的聊天 UI 和会话查看
- ▶ 官方路径也是这个套路
20 行: 脚本 → 带存储/追踪的服务

这三颗雷，排掉就是经验

日志写着 OpenAI API?

别慌，没配错。

DeepSeek 适配器复用 OpenAI 兼容客户端，
底层日志沿用了这个名字，请求仍走
DeepSeek 官方接口。

import 即运行

示例在模块顶层调模型。

哪怕只是 import 这个文件，也会立刻发请求、烧 token。正式代码要包进 main。

AgentOS 起不来

先查端口占用：

```
lsof -nP -iTCP:7777 -sTCP:LISTEN
```

被占了就停掉旧进程或改端口。

检查点四连问

01 用一句话说清 Agent 是什么？

提示：stateful control loop / stateless model

02 description、instructions、expected_output 分别影响什么？

提示：人设 / 规矩 / 格式提示（不校验）

03 想要稳定、可校验的结构化输出，该用哪个参数？

提示：不是 expected_output

04 什么场景下 print_response 必须换成 aprint_response？

提示：和异步工具有关

本章一句话

Agent = 模型 + 指令 + (可选的) 工具 外面套一个控制循环

下一章 · ch2 工具调用：让 Agent 从「会回答」到「会行动」。

按 [←](#) 返回复习 · [Home](#) 回到封面